

The 16th International Conference on Ambient Systems, Networks and Technologies (ANT)
April 22-24, 2025, Patras, Greece

Eclipse SUMO and Unity 3D integration for emission studies based on driving behaviour in virtual reality environment

Viktor Nagy^{a,*}, Csaba J. Májer^a, Sámuel Lepold^a

^a*Széchenyi István University - Faculty of Architecture, Civil Engineering and Transport Sciences - Department of Transport, Egyetem tér 1., Győr 9026, Hungary*

Abstract

This paper explores the development of a traffic simulator that harnesses the power of OpenStreetMap (OSM) data, integrating it into a unified framework using Blender, SUMO (Simulation of Urban MObility), and Unity 3D. The project aims to provide a realistic and versatile platform for simulating complex transportation scenarios for urban planning, traffic management, and virtual environment creation. The process begins with the extraction and preprocessing of OSM data, ensuring the representation of road networks and geographic features. Blender, a 3D modelling software, is used to visualise and enhance the environment. The resulting 3D models are loaded into Unity. SUMO – a microscopic traffic simulation software – plays a role in modelling vehicle dynamics, traffic flow, and interactions within the simulated environment. Via an integration hub, bringing together the visuality of Unity 3D environment and SUMO's precise traffic simulation. Through this collaboration, the simulator offers real-time interaction and analysis. With the help of SUMO's free emissions module, the pollution resulting from driving behaviour can be measured not only at the individual level but also the impact on the network as well. The combination of OSM data, SUMO, Blender, and Unity 3D provides a powerful and free tool to study and optimise urban mobility, ultimately contributing to the development of smarter and more sustainable transportation systems.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer review under the responsibility of the scientific committee of the Program Chairs

Keywords: SUMO; emission; driving behaviour; virtual reality; simulator

1. Introduction

Climate change has become one of the most pressing issues today. Countries and alliances are setting new milestones to stop global warming. Based on the Clean and Sustainable Mobility Policy of the European Council, transport is responsible for a quarter of the EU's greenhouse gas emissions, with passenger car transport accounting for 71 percent. On this basis, individual passenger car transport is responsible for almost 18 percent of EU greenhouse gas

* Corresponding author. Tel.: +36-96-503-400.

E-mail address: nagy.viktor@sze.hu

emissions. For this reason, it is of paramount importance to examine how individuals influence their environment through their transport. Another serious external cost of private transport that raises sustainability issues is related to land use. In traffic simulations, different software uses different parameters to simulate driver behavior. These values can influence the resulting queuing lengths of the simulation by up to 10 percent, as shown in the study of Szinyéri et al.[16]. In both cases, it is true that the driving behavior of vehicle users influences the simulation outcome. Furthermore, drivers shape not only the driving of their own vehicle, but also the driving behavior of their environment due to following distances and extra emissions caused by braking and acceleration.

The potential of car simulators has long been explored, most of which enable road safety tests or can be used in driver training to test live situations[12]. There have been several studies on the potential and limitations of transport models, such as the paper of Azlan and Rohani[14] or the study of Dorokhin et al.[2]. Transport modeling software such as SUMO (Simulation of Urban MObility) is constantly evolving and it is a commonly used tool for transport research[9][1]. Its realization has not started today. Griggs and Shorten developed a method to detect vehicle position using a mobile phone and place it in a SUMO environment[5], followed by further hardware-in-the-loop simulation research[4]. Prevati and Mastinu also addressed the issue of connected vehicles and autonomous vehicles in a SUMO environment[15]. Hu et al. created an approach in which they tested autonomous vehicles in SUMO with digital twin technologies[8]. Kusari et al. have also investigated autonomous vehicles, combining SUMO software with the OpenAI Gym environment[10]. Garrido et al. have created a virtual environment using Unity 3D software to refine pedestrian behaviour and perform related studies[3]. In their research, they connected the SUMO environment to the 3D space using the TraCI API. There are some examples of bicycle simulations[6], but it is clear that studies have recently shifted towards research on self-driving cars. Vehicle emissions are addressed in the paper by Tettamanti et al., where the emissions of hybrid electric vehicles in a SUMO environment were investigated[17]. Yunfei et al. validated the operation of integrating a simulation program and a driving simulator, which is capable of measuring the drivers pollutant emissions[7].

Open-source products are gaining ground in research, but are also increasingly used by industry. This study has also tried to use open source or free products for research, so that the ideas raised here can be made freely available to others for further research. In this research, OpenStreetMap, SUMO[11], Blender, Unity 3D, and Python[19] were used. The development was based on the BME Automated Drive Lab project[18], which has been extended with several individual developments in the current project (SUMITY). It consists of two main elements: The 3D visualization, which is based on the OpenStreetMap of Győr, Hungary, and the Python code, which connects the different software and is also responsible for the control.

Previous studies and simulators focused on individuals, examining their emissions. The newly developed simulator, based on the SUMO traffic simulation program, combines both individual emissions and those projected onto the entire network. In its current state, the software can analyze the impact of an individual traveler on the entire network. Additionally, it can be further expanded with additional modules. Real reactions to real environments can help us understand how travelers behave in different traffic situations and highlight certain patterns. This allows us to identify which groups of travelers should be targeted, what type of education would be most effective, and how specific infrastructure designs can influence not only traffic safety but also pollutant emissions.

This study is a first step in a larger research project to find answers to the question of how much impact an individual has on the emissions of the entire network and how the calibration of each model could be more accurate to obtain more precise results. In the first phase of the research, an investigation was conducted into whether it is possible to create a simulator of this type, which connects the SUMO traffic simulation program with the Unity3D game development software.

2. Principle of SUMITY

The program consists of seven Python files. The flow chart in Fig. 1 shows the programming principles of the source code.

- **Main.py:** The main entry point of the application that initializes and runs the simulation.
- **Control.py:** Handles user inputs for controlling a simulated vehicle via the keyboard, Xbox controller, or steering wheel.

- SUMO_vehicle.py: Manages and interacts with vehicles in the SUMO simulation.
- TCP_server.py: Maintain the TCP server for communication between the Python backend and the Unity frontend.
- TrafficLight.py: Handles interaction with and state updates of traffic lights in the setup.
- TrafficSimulator.py: Acts as a bridge between the SUMO simulation and the Unity visualization.
- Unity.py: Supports the translation of SUMO simulation elements to Unity 3D recognizable forms.

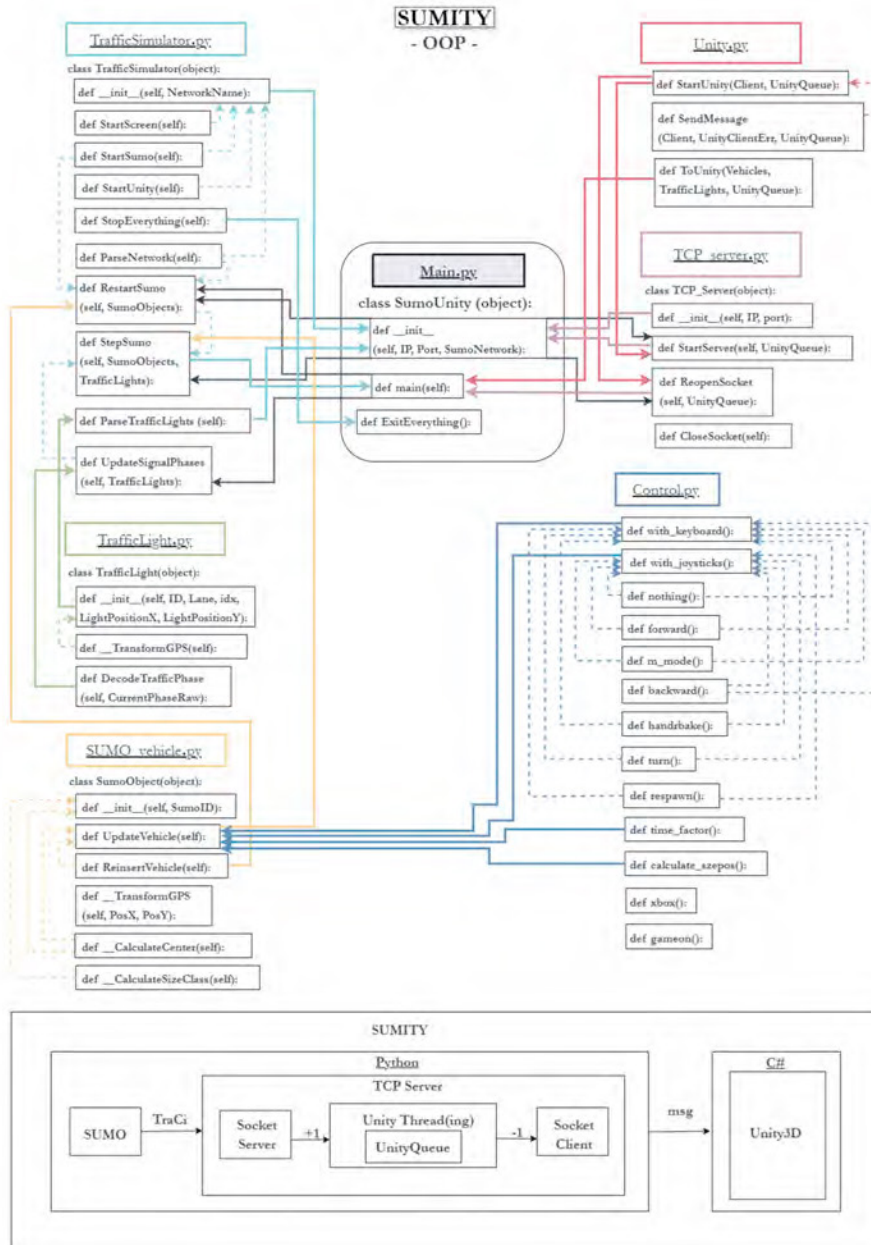


Fig. 1. The flowchart of the program.

Together, these modules develop a complete traffic simulation system with user interaction, multiple vehicle handling, traffic lights implementation, and a TCP server that facilitates communication between the SUMO traffic simulator and the Unity visualization environment. The step-by-step explanation is the following:

Step 1 - Launching the simulation: The execution of this project begins with `Main.py`. It is responsible for setting up the connection to the Traffic Simulation (handled by SUMO) and the visualization software (handled by Unity3D). Once these connections are configured, `Main.py` then begins the traffic simulation.

Step 2 - Traffic simulation: The traffic simulation logic is handled by `TrafficSimulator.py`, which is connected to SUMO and manages the simulation time step. It includes the retrieval of vehicle and traffic light objects from SUMO in a manageable Python format.

Step 3 - Handling traffic lights and vehicles: `TrafficLight.py` provides an interface for interaction with and updating the state of traffic lights in the simulation. The `TrafficLight` class allows for the decoding of traffic light phases (red, yellow, green, etc.) and the calculation of GPS positions from SUMO's X-Y coordinate system. The `SUMO_vehicle.py` module is used to interact with the vehicles. The `SumoObject` class sets up properties for each vehicle, updates vehicle states based on user input, and handles other maintenance tasks such as reinserting lost vehicles back into the simulation.

Step 4 - User input: The `Control.py` module is responsible for handling user inputs to control a simulated vehicle. This can be done through several inputs, such as the keyboard, Xbox controller, and steering wheel. The inputs include actions for the vehicle, such as gas, brake, steer, handbrake, and respawn. It also takes care of updating the time factors and others based on the user input.

Step 5 - Connecting to Unity: `TCP_server.py` opens a TCP server to communicate with a Unity client. Data about vehicles and traffic lights, updated per the SUMO simulation timestep, are sent over this TCP connection to the Unity application. The `Unity.py` script then processes the incoming TCP data and converts the information into a format that Unity 3D can use to render the vehicle objects and traffic light states.

Step 6 - Updating objects: The `SumoObject` class in `SUMO_vehicle.py` updates the SUMO vehicles, and the `TrafficLight` class in `TrafficLight.py` decodes the traffic light phases.

Step 7 - Looping through steps: Steps 2 to 6 are then looped for each timestep in the simulation, ensuring that the vehicles move and the traffic light phases change over time and reflecting any user input to control the selected vehicle.

Step 8 - Exiting: Once the simulation is completed or manually stopped, all connections are closed, and the program is exited.

3. Control system

It is made up of different methods that allow one to reproduce the kinematics of real vehicles as accurately as possible. The `with_keyboard()` function allows the user to control the simulated vehicle with keyboard input. The `with_joysticks()` is responsible for processing and implementing input signals from various controllers in the control process. Currently, our program supports an Xbox controller and a Gameon steering wheel, but we can extend this method to most known controllers. For each type of controller, it maps the game actions (gas, brake, steer, handbrake, respawn, and 'M' mode) to specific joystick actions.

The `nothing()`, `forward()`, `m_mode()`, `backward()`, `handbrake()`, `turn()`, `respawn()`, `time_factor()` and `calculate_szeapos()` methods manipulate the global car control variables depending on the input received. They simulate the car's behavior in response to the inputs like moving forward, backward, turning, applying handbrake, respawn, and calculating the new position of the vehicle.

The `nothing()` function is intended to represent the state of a vehicle when the driver does not apply any gas torque to the vehicle. That is the aerodynamic drag of vehicles and the engine brake.

The `forward()` function is used to generate the actual acceleration value of the controlled vehicle. To determine the arithmetic values, we used the measurement results of Melegh (2004). In his experiment, he determined the acceleration curve of a Citroen Berlingo as a function of time(t), from which we mapped the value of the `SZE_SPEED`:

$$SZE_SPEED = \sqrt{1400 * (t + 3)} - 64.81 \quad (1)$$

The simulation is also prepared for the mode selection function typical of today's modern vehicles. This is covered by the `m_mode()` function, which currently simulates the sporty driving style. However, the same principle can also be used to implement an ECO mode.

The `backward()` performs the braking function of the vehicle. When the speed reduction reaches 0 km / h, the vehicle is reversed.

The `handbrake()` represents the handbrake, which is defined as emergency braking, that is 7.8 m/s² deceleration[13]

The specificity of the simulation is its ability to reproduce the operating characteristics of a speed-dependent power steering. As the vehicle speed increases, the length of the turning arc changes. The Eq(2) formula was used:

$$Route_Length = \frac{CAR_TURN_RADIUS * \pi * Length_Factor}{|SZE_STEERING_ANGLE| * 2} \quad (2)$$

where:

- `CAR_TURN_RADIUS` is originally specified as 5 meter,
- `SZE_STEERING_ANGLE` is the steering angle in the range -1 to +1 and
- `Length_Factor` was calculated with the following formula:

$$Length_Factor = 0.00075 * SZE_SPEED^2 + 0.00375 * |SZE_SPEED| + 1 \quad (3)$$

The `respawn()` function is responsible for setting the vehicle's XY coordinate to its initial position and placing it at a specific point on the map.

The `time_factor()` function is used to synchronize time in our system. In the source code, we defined the time intervals at which the data should be collected, which we also synchronized with the SUMO program. The simulation currently works with a frame rate of 40 FPS.

The `calculate_szeupos()` function is responsible for the correct placement of the controlled vehicle. It works with the X, Y coordinates, and the alignment angle. The Python program performs a continuous calculation to update these three attributes of the vehicle. The values are updated by signals from the input devices used by the user. In SUMO, the continuous movement of the vehicle is done by placing it in the new XY coordinates, which is performed by the "`traci.vehicle.moveToXY()`" function of the `SUMO_vehicle.py` module.

The `xbox()` and `gameon()` methods provide mapping of controls for the Xbox controller and Gameon steering wheels. They are not to be used directly to get input.

In general, `Control.py` provides the main functionality for parsing player input (from keyboard, Xbox controller, or steering wheel) and updating the state. The data from this file would then need to be communicated back to the Unity framework, where it would be used to update the visuals.

4. Visualization

The real world mapping was done using OpenStreetMap (OSM), Blender, and Unity 3D. The information retrieved from the OSM network was the basis for the transport network, which was represented in the SUMO environment by the SUMO 'netconvert' command and then cleaned up in the Netedit graphical environment. The OSM network was also the basis for the buildings for which the OSM2World application was used. The roads were also created in this

way in the 3D environment. The '.obj' files were then cleaned up in the Blender software. Blender is a free, open-source 3D software that enables 3D modeling. Due to its flexibility and expandability, it is used in various industries. In this case, the software was used to clean up the 3D environment elements, such as buildings and trees generated from the OSM import, and unnecessary 3D elements were deleted. These 3D objects were then used to construct the virtual environment in Unity 3D. In this study, only a small section of the city was included in the virtual environment to test its usability. Later on, the 3D model was transferred from Blender to Unity 3D, where the 3D environment with the driver's car, the buildings (without skin for the moment) and the roads was added by the visualization packages. Finally, the speedometer was created in Unity 3D software, which gives the driver accurate information on current speed. The SUMITY in operation is shown in Fig. 2.



Fig. 2. The SUMITY environment in early stages.

At the level of the software, Unity is responsible for the possibility of switching between different views, but the control is already done through a Python code, however, visualization in a 3D environment is necessary for accurately measuring the visual experience and reactions as observed in real-life scenarios. The SUMO program interprets vehicles as rectangles with a length of 4.25m and a width of 1.5m. The vehicle displayed in Unity has the same size parameters. Through the connection of the two software, the vehicle's position is determined as a single point, aligned with the center of the front bumper, taking the current angle into account. The vehicles take each other's positions into account and react to one another. However, physical collisions do not occur; the simulated vehicles do not crash into each other and follow traffic rules properly. In contrast, the controlled vehicle can approach other vehicles without limitations, but collisions are not simulated. This is because such situations do not provide meaningful insights into emissions, as they represent an unusual traffic scenario.



Fig. 3. Simulator test in SUMITY.

The environment creation is still in its early stages, so a demo version has also been created using the free Unity assets. The available simulator has high performance, and no lag is experienced during the execution of the developed software. In the initial tests, the demo network had seven incoming lanes, with vehicles being loaded every two seconds. The Unity 3D environment does not render all vehicles, buildings, and network elements simultaneously—only

the elements visible in the vicinity of the vehicle are displayed. The demo city, in its current structure and level of detail (including textured buildings, network infrastructure, vehicles, road defects, benches, trees, lamp posts, and lighting), ran smoothly without any lag and maintained a stable 40 FPS with a 7680x1440 screen resolution. Fig. 3 shows the test run in the demo city.

The recording of various parameters is done through SUMO, utilizing its built-in output functions. Currently, the emission output is being stored and processed. The processing is handled by a Python program (Fig. 4), which in its current form can separately analyze emissions of the controlled vehicles or the entire network for five types of pollutants (CO₂, CO, HC, NO_x, PM_x), filtered for a specific time period. As a further development, work has begun to visualize on a map these values, which in the future will also take gas dispersion and buildings into account.



Fig. 4. The emission modul.

5. Conclusions

In the initial phase of the research, a simulation software system has been created and will be developed to carry out further studies. During network generation, it became apparent that the sumo netconvert option generated many redundant edges and junctions during the OSM file conversion, which had to be manually corrected. Similarly, converting to a 3-dimensional environment involves a lot of unnecessary edit points and faces for edges, which greatly increases the size of the resulting file and unnecessarily slows down the system. By manually editing the model, this can be reduced to a fraction of the size (about a tenth). Another issue that arises may be the different network in this case between the manually edited '.net.xml' file and the '.blend file'. The future goal is to develop this method to ensure that the SUMO and 3D road network models do not differ. Thus, we are working on the development of a procedure that can be built directly from the road network system of the existing SUMO model and transform it into a 3-dimensional object. A further problem could be the fitting of buildings with appropriate skins, which would require the photography of actual facades on a building-by-building basis, which would be an unnecessarily large amount of work. Instead, we would like to create a library of real photos, from which the building skins would be randomly selected. We see further potential in the photogrammetric processing of the Google Earth application.

Integration of the SUMO environment into the system will allow for various emission tests according to the HBEFA4.2 standard (CO₂, CO, HC, NO_x, PM_x), both at the individual level and for the entire network. Furthermore, it can be used for tests in which monitoring driver reaction times, peripheral vision, and eye movements can provide a detailed picture of driving skills and distractions in the traffic environment that can influence accident risk. Monitoring vital signs such as heart rate and cognitive ability helps identify stress levels and environmental distractions in different traffic situations. By measuring driver behavior, traffic models can become more accurate and realistic. The integration of hardware tools, such as simulators, offers potential for improvement. In addition, there is the possibility to adapt various features of passenger vehicles, such as cruise control, lanekeeping, driving mode switch, adaptive cruise control (ACC), or table recognition. The appropriate placement of the traffic lights in the simulation is necessary. The inclusion of topography and a more realistic representation of the current environment (e.g. building skins) can result in more lifelike and accurate simulation software.

Acknowledgements

Supported by the EKÖP-24-3-I-SZE-96 university research fellowship program of the Ministry for Culture and Innovation from the source of the National Research, Development and Innovation Fund.



References

- [1] Behrisch, M., Bieker, L., Erdmann, J., Krajzewicz, D., 2011. Sumo - simulation of urban mobility an overview, in: SIMUL 2011: The Third International Conference on Advances in System Simulation, pp. 63–68.
- [2] Dorokhin, S., Artemov, A., Likhachev, D., Novikov, A., Starkov, E., 2020. Traffic simulation: an analytical review. IOP Conference Series: Materials Science and Engineering 918, 012058. doi:[10.1088/1757-899X/918/1/012058](https://doi.org/10.1088/1757-899X/918/1/012058).
- [3] Garrido, D., Jacob, J., Silva, D.C., Rossetti, R.J.F., 2021. Pedestrian simulation in sumo through externally modelled agents. European Conference on Modelling and Simulation, 111–118doi:[10.7148/2021-0111](https://doi.org/10.7148/2021-0111).
- [4] Griggs, W.M., Ordóñez-Hurtado, R.H., Crisostomi, E., Häusler, F., Massow, K., Shorten, R.N., 2015. A large-scale sumo-based emulation platform. IEEE Transactions on Intelligent Transportation Systems 16, 3050–3059. doi:[10.1109/TITS.2015.2426056](https://doi.org/10.1109/TITS.2015.2426056).
- [5] Griggs, W.M., Shorten, R.N., 2013. Embedding real vehicles in sumo for large-scale its scenario emulation, in: 2013 International Conference on Connected Vehicles and Expo (ICCVE), pp. 962–963. doi:[10.1109/ICCVE.2013.6799942](https://doi.org/10.1109/ICCVE.2013.6799942).
- [6] Hammami, A., Borsos, A., Sandor, A.P., 2023. Bike simulator applications in traffic engineering: a comprehensive review, in: I. Transport safety conference, pp. 61–72.
- [7] Hou, Y., Zhao, Y., Hulme, K., Huang, S., Yang, Y., Sadek, A., Qiao, C., 2014. An integrated traffic-driving simulation framework: Design, implementation, and validation. Transportation Research Part C: Emerging Technologies 45. doi:[10.1016/j.trc.2014.02.004](https://doi.org/10.1016/j.trc.2014.02.004).
- [8] Hu, X., Li, S., Huang, T., Tang, B., Huai, R., Chen, L., 2024. How simulation helps autonomous driving: A survey of sim2real, digital twins, and parallel intelligence. IEEE Transactions on Intelligent Vehicles 9, 593–612. doi:[10.1109/TIV.2023.3312777](https://doi.org/10.1109/TIV.2023.3312777).
- [9] Krajzewicz, D., 2010. Traffic Simulation with SUMO – Simulation of Urban Mobility. Springer New York, New York, NY. pp. 269–293. URL: https://doi.org/10.1007/978-1-4419-6142-6_7, doi:[10.1007/978-1-4419-6142-6_7](https://doi.org/10.1007/978-1-4419-6142-6_7).
- [10] Kusari, A., Li, P., Yang, H., Punshi, N., Rasulis, M., Bogard, S., LeBlanc, D.J., 2022. Enhancing sumo simulator for simulation based testing and validation of autonomous vehicles, in: 2022 IEEE Intelligent Vehicles Symposium (IV), pp. 829–835. doi:[10.1109/IV51971.2022.9827241](https://doi.org/10.1109/IV51971.2022.9827241).
- [11] Lopez, P.A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., Wiessner, E., 2018. Microscopic traffic simulation using sumo, in: 2018 21st International Conference on Intelligent Transportation Systems (ITSC), pp. 2575–2582. doi:[10.1109/ITSC.2018.8569938](https://doi.org/10.1109/ITSC.2018.8569938).
- [12] Maroto, J., Delso, E., Felez, J., Cabanellas, J.M., 2006. Real-time traffic simulation with a microscopic model. IEEE Transactions on Intelligent Transportation Systems 7, 513–527. doi:[10.1109/TITS.2006.883937](https://doi.org/10.1109/TITS.2006.883937).
- [13] Melegh, G., 2004. Automotive expertise (in Hungarian). Maróti Könyvkereskedés és Könyvkiadó Kft.
- [14] Nor Azlan, N.N., Md Rohani, M., 2018. Overview of application of traffic simulation model. MATEC Web of Conferences 150, 03006. URL: <https://doi.org/10.1051/mateconf/201815003006>, doi:[DOI:10.1051/mateconf/201815003006](https://doi.org/10.1051/mateconf/201815003006).
- [15] Prevati, G., Mastinu, G., 2023. Sumo roundabout simulation with human in the loop, in: SUMO Conference Proceedings, p. 29–40. doi:[10.52825/scp.v4i.211](https://doi.org/10.52825/scp.v4i.211).
- [16] Szinyéri, M., Rékai, G., Nagy, V., 2023. Determination of certain initial calibration parameters of traffic simulation for the hungarian urban environment in vissim microsimulation software (in hungarian), in: XIII. International Conference on Transport Sciences, pp. 496–508.
- [17] Tettamanti, T., Okolie, C.A., Ormandi, T., Varga, B., 2024. A python tool for sumo traffic simulation to model hybrid electric vehicles (hev) for comprehensive emission analysis doi:[10.13140/RG.2.2.33734.69443](https://doi.org/10.13140/RG.2.2.33734.69443).
- [18] Tettamanti, T., Szalai, M., Vass, S., Tihanyi, V., 2018. Vehicle-in-the-loop test environment for autonomous driving with microscopic traffic simulation, in: 2018 IEEE International Conference on Vehicular Electronics and Safety (ICVES), pp. 1–6. doi:[10.1109/ICVES.2018.8519486](https://doi.org/10.1109/ICVES.2018.8519486).
- [19] Van Rossum, G., Drake Jr, F.L., 2011. Python reference manual. Network Theory Ltd.